# Daily Paper Review: Intern Lumina U2: A Multi-Codebook Diffusion Large Language Model

Auto-generated Report

October 18, 2026

## 1 Paper Summary

### 1.1 Problem and Motivation

Unified models that handle both visual understanding and visual generation face a fundamental representational tension. Autoregressive (AR) models handle text naturally but require separate continuous diffusion decoders for image generation, creating optimization conflicts between modalities. Discrete unified approaches (e.g., Janus-Pro, Show-o) use a single codebook per spatial position, compressing each visual location into one discrete code from a vocabulary of ∼8192 entries—a severe bottleneck that cannot capture the full richness of texture, color, and semantics. Hybrid continuous–discrete approaches maintain two distinct generative objectives with different loss landscapes, complicating training.

Yu et al. (arXiv:2610.20818) propose Intern Lumina U2, a unified diffusion large language model that resolves these issues by introducing **multi-codebook discrete visual tokens**. Each spatial position is represented by $K$=8 codes from 8 independent codebooks (each with $V$=4096 entries), yielding an effective representational capacity of $4096^8 \approx 10^{29}$ states per position—compared to just $V$ states with a single codebook. Crucially, all modalities (text, images, video, 3D) share the same masked discrete diffusion objective, eliminating the optimization mismatch of hybrid approaches while dramatically expanding visual fidelity.

### 1.2 Method

**Multi-Codebook Visual Tokenization (AToken).** The visual tokenizer, AToken, is initialized from a pretrained SigLIP2 encoder and uses **Finite Scalar Quantization (FSQ)** over $K$=8 codebooks. Each spatial position produces 8 independent discrete codes $(c_1, c_2, \ldots, c_8)$. Importantly, AToken provides a **4D unified** representation: images, videos, and 3D assets all map into the same discrete space, enabling a single tokenizer for all visual modalities. The tokenizer is *frozen* during all LLM training stages.

**Multi-Codebook Embedding.** Each visual position receives $K$=8 independent embedding tables $(E_1, \ldots, E_K)$. The embeddings are concatenated and projected:

$$x = W_{\text{in}}\big[E_1(c_1); E_2(c_2); \ldots; E_K(c_K)\big] \tag{1}$$

where $W_{\text{in}} \in \mathbb{R}^{d \times Kd}$ maps the $Kd$-dimensional concatenation back to the model's hidden dimension $d$. This preserves the original sequence length—the $K$ codebooks are folded into the embedding dimension rather than multiplying the sequence length, avoiding a $K\times$ increase in computational cost.

**Multi-Codebook Autoregressive Head.** Rather than predicting all $K$ codes independently (which ignores inter-codebook dependencies), the output head factorizes the joint distribution autoregressively along the codebook (depth) dimension:

$$p(\mathbf{c} \mid h) = \prod_{k=1}^{K} p(c_k \mid h, c_{<k}) \tag{2}$$

This is implemented as a lightweight 6-layer Transformer that operates along the depth axis. At position $k$=1, the head receives a projection of the backbone hidden state $h$ plus a learnable embedding; at $k \geq 2$, it receives the embedding of the preceding ground-truth code (teacher forcing during training). Separate classifiers $W_k^{\text{out}} \in \mathbb{R}^{4096 \times d}$ produce logits for each codebook. Ablations confirm that this autoregressive factorization significantly outperforms independent parallel prediction of all $K$ codes.

**Backbone Architecture.** Built upon LLaDA2.0-mini, the backbone has 16.9B total parameters with ∼1.4B activated per token. It uses 20 Transformer layers: 1 dense FFN layer followed by 19 MoE layers (256 routed experts, 8 activated per token, plus 1 shared expert). The visual input/output interfaces add 0.65B parameters. The backbone was obtained by converting a pretrained autoregressive MoE model into a block diffusion model following the LLaDA2.0 methodology.

**Masked Discrete Diffusion Process.** The diffusion process is **absorbing-state masked diffusion**, where corruption replaces tokens with a `[MASK]` token. For text, block-wise masking (block length $B$=32) is applied with a cosine schedule: $\rho = \cos(\frac{\pi}{2}u)$, $u \sim \mathcal{U}(0, 1)$, with a 3% chance of fully masking each block. For visual tokens, a single mask ratio $\rho$ (drawn from the same cosine schedule) applies across all spatial positions simultaneously.

The reverse process (denoising) proceeds as:

1. The model predicts logits for all masked positions in parallel
2. Tokens are sampled via temperature-scaled Gumbel: $\hat{x}_i = \arg\max(\text{logits} + \text{Gumbel} \cdot \tau)$
3. Confidence is computed: $\sigma_i = \text{softmax}(\text{logits})[\hat{x}_i]$
4. The $\nu_s$ most confident positions are committed (unmasked) per step

For text, decoding proceeds block-by-block (left-to-right) with $S$ iterative unmasking steps per block. For visual tokens, all positions are decoded jointly through full spatial diffusion.

**Training Objective.** The loss averages text and visual cross-entropies over masked positions:

$$\mathcal{L} = \frac{\sum_{i \in \mathcal{T}} \ell_{\text{txt}}(i) + \sum_{j \in \mathcal{V}} \ell_{\text{vis}}(j)}{|\mathcal{T}| + |\mathcal{V}|} + 10^{-3} \cdot \mathcal{L}_{\text{balance}} \tag{3}$$

where $\ell_{\text{txt}}(i) = -\log p(x_i^{\text{gt}} \mid x_{\text{masked}})$ is the standard cross-entropy for masked text tokens, and $\ell_{\text{vis}}(j) = \frac{1}{K}\sum_{k=1}^{K} -\log p(c_k^{\text{gt}} \mid h, c_{<k}^{\text{gt}})$ averages over the $K$ codebooks per visual position. The $1/K$ normalization ensures each visual position receives equivalent weight to each text token. $\mathcal{L}_{\text{balance}}$ is the MoE load-balancing loss.

**Plateau-Halton Sampling (PHV).** A training-free inference acceleration technique achieves **4.45× end-to-end speedup** for image generation. It monitors the exponential moving average of maximum confidence across steps; when the slope drops below 0.01 ("plateau"), it forces early commits rather than waiting for the schedule threshold. A drift-triggered cache refresh mechanism tracks hidden-state $L_2$-norm statistics and recomputes the backbone cache when drift exceeds 5% tolerance.

### 1.3 Results

With only ∼1.4B activated parameters, Intern Lumina U2 achieves:

**Image Understanding:** First place on 6 of 10 benchmarks vs. same-backbone LLaDA2.0-Uni, with improvements on 8 of 10. Notable: HallusionBench 64.6 (vs. 50.2 for LLaDA2.0-Uni), MMMU 52.2 (vs. 50.1), MathVista 68.0, ChartQA 86.2.

**Image Generation:** GenEval 89 (matching LLaDA2.0-Uni), DPG-Bench 87.79 (vs. 87.76). TIIF-Bench (long prompts) 87.1. Generation quality is maintained while significantly improving understanding.

**Video Understanding:** Video-MME 52.6, MLVU 62.1, MVBench 61.8—superior to both diffusion-based (LLaDA-V) and dedicated video models (VideoLLaMA2) on all three benchmarks.

**3D Understanding:** 3D MM-Vet 50.5, outperforming specialist models ShapeLLM and PointLLM.

**Image Editing:** ImgEdit 3.82/5, GEdit-Bench EN 6.06/10, CN 5.95/10.

The model was trained on ∼1.05T tokens across 5 stages on 1,024 Ascend 910B NPUs (and a parallel 650B-token NVIDIA run).

### 1.4 Limitations

(1) **Frozen tokenizer**: AToken is frozen during all LLM training stages; any tokenizer deficiencies cannot be corrected during end-to-end training, and no reconstruction FID numbers are reported to assess tokenizer quality independently. (2) **Sequential codebook prediction**: The autoregressive head must predict $K$=8 codes sequentially per position during inference, creating a per-position depth bottleneck even though spatial positions are processed in parallel. (3) **No text-only evaluation**: Despite building on LLaDA2.0 (a text diffusion backbone), no text-only generation benchmarks (perplexity, LAMBADA, HellaSwag) are reported, leaving unclear whether multimodal training degrades text capabilities. (4) **Near-saturated generation metrics**: GenEval scores are identical to LLaDA2.0-Uni (89), making the multi-codebook contribution difficult to isolate on generation benchmarks. (5) **Limited comparison scope**: No comparisons with state-of-the-art dedicated generation models (DALL-E 3, FLUX) or large AR multimodal models (GPT-4V, Gemini) at similar active parameter counts. (6) **Heuristic mask schedule**: The cosine schedule $\rho = \cos(\frac{\pi}{2}u)$ is empirically chosen without optimality justification. (7) **No human evaluation** of generation quality.

### 1.5 Relevance to Research

Intern Lumina U2 directly extends the diffusion LLM line central to Topic T4. The paper builds on LLaDA2.0-Uni (Review #38), which unified multimodal understanding and generation under masked discrete diffusion but used single-codebook visual tokens. The multi-codebook extension addresses the representational bottleneck that limited visual fidelity in prior discrete diffusion approaches like Omni-Diffusion (Review #7) and Cubic Discrete Diffusion (Review #14). The autoregressive codebook head introduces a novel *within-position depth dimension* for discrete diffusion, complementing the *spatial* masked diffusion process—this is conceptually related to the depth-wise factorization explored in Subliminal Clocks (Review #99), which modeled latent time within diffusion LLMs. The PHV sampling acceleration technique (4.45× speedup) connects to the inference efficiency work in SpecFold (Review #122) and S2D2 (Review #19), which accelerated diffusion LLM decoding via speculative methods; PHV achieves comparable speedups through confidence-based early commitment rather than parallel drafting. The FSQ-based tokenization contrasts with the VQ-VAE approach used in most prior discrete diffusion LLMs like Sumi (Review #76), offering a codebook-collapse-free alternative that naturally extends to multiple codebooks. For quantization (T1), the multi-codebook structure presents

an interesting challenge: SoloQ (Review #123) quantized single-codebook dLLMs, but the 6-layer autoregressive head introduces a new quantization target where errors in early codebook predictions could cascade through the depth dimension.

# 2 Follow-up Research Directions

## 2.1 Direction 1: Parallel Multi-Codebook Decoding via Depth-Wise Diffusion

**Gap:** Intern Lumina U2's autoregressive codebook head predicts $K$=8 codes sequentially per spatial position, creating an $O(K)$ depth bottleneck that compounds with the spatial iterative unmasking steps. While the autoregressive factorization captures inter-codebook dependencies, it assumes a strict left-to-right ordering ($c_1 \to c_2 \to \ldots \to c_8$) that may be unnecessarily constraining. No existing method applies diffusion-style parallel decoding along the codebook (depth) dimension.

**Approach:** Replace the autoregressive codebook head with a **depth-wise masked diffusion** process: treat the $K$=8 codes at each spatial position as a short sequence and apply the same absorbing-state masked diffusion used spatially. During inference, all $K$ codes start as `[MASK]` and are iteratively unmasked over $S_d$ depth steps (with $S_d \ll K$ sufficient for $K$=8). This enables parallel prediction of multiple codebook entries per step, reducing the depth bottleneck from $O(K)$ to $O(S_d)$. The key insight is that masked diffusion naturally handles dependencies without requiring a fixed ordering—if $c_3$ and $c_7$ are independent conditioned on $c_1$ (already committed), they can be predicted simultaneously. The confidence-based commitment from Intern Lumina U2's spatial diffusion applies directly to the depth dimension. This creates a *doubly-diffusion* architecture: spatial diffusion across positions × depth diffusion across codebooks. Connect to SpecFold (Review #122), whose multi-branch speculation could be adapted to speculate across codebook positions rather than spatial positions, yielding a speculative depth-decoding strategy.

**Feasibility:** High. The masked diffusion machinery already exists in the backbone and can be reused for the depth dimension with minimal architectural changes. The main risk is that $K$=8 may be too short for iterative diffusion to outperform autoregressive prediction (diffusion benefits scale with sequence length). Empirically, 2–3 depth diffusion steps may suffice for $K$=8, yielding a $\sim$3× speedup in the depth dimension. The PHV early-commitment heuristic naturally extends to depth-wise decoding.

## 2.2 Direction 2: Adaptive Codebook Count via Difficulty-Aware Token Allocation

**Gap:** Intern Lumina U2 uses a fixed $K$=8 codebooks for every spatial position regardless of visual complexity. Homogeneous backgrounds need far less representational capacity than intricate textures or object boundaries. The paper provides no analysis of how many codebooks are "actively used" (i.e., carry significant information) at different spatial locations. This uniform allocation wastes compute on simple regions and may underserve complex ones.

**Approach:** Introduce **position-adaptive codebook allocation**: a lightweight predictor estimates the "visual complexity" $k_j \in \{1, 2, \ldots, K\}$ for each spatial position $j$ based on the backbone's intermediate representation, and only the first $k_j$ codebooks are predicted for that position. The remaining $K - k_j$ codes are set to a default "null" value. This is analogous to early exit in autoregressive models—simpler visual regions terminate codebook prediction early. The complexity predictor can be trained with a sparsity-inducing loss that penalizes the average $k_j$ across positions, creating a compute–quality tradeoff controlled by a single hyperparameter. This connects to LatentLoop (Review #127), which discovered that different loop iterations deserve asymmetric compute allocation; similarly, different spatial positions may

deserve asymmetric codebook depth. The approach also parallels the confidence-based early stopping in Intern Lumina U2's own PHV sampling, extending the "commit when confident" principle from the temporal (denoising steps) dimension to the depth (codebook) dimension. For high-resolution generation (e.g., $1024 \times 1024$), adaptive allocation could reduce the average $K$ from 8 to $\sim$4–5 across positions, yielding $\sim$40–50% reduction in depth-decoding compute.

**Feasibility:** Medium-high. The complexity predictor adds negligible parameters ($d \to K$ projection per position). The main challenge is that the frozen AToken tokenizer was trained with fixed $K$=8; null codes would need to be mapped to a "no-op" embedding. An alternative is to train the AToken with variable $K$ from the start, but this requires re-training the tokenizer.

### 2.3 Direction 3: Cross-Modal Codebook Transfer for Text–Visual Diffusion Alignment

**Gap:** In Intern Lumina U2, text tokens live in a single-codebook space (vocabulary of 102,400) while visual tokens use 8 codebooks of 4096 entries each. This asymmetry means the diffusion process operates over fundamentally different representational structures for text vs. vision, despite sharing the same backbone and masking objective. The paper notes that the $1/K$ normalization ensures equal loss weighting, but does not investigate whether text and visual representations align in the shared backbone hidden space. Cortex (Review #126) showed that context-to-memory distillation can transfer representations across formats; a similar principle could align text and visual codebook representations.

**Approach:** Extend the multi-codebook paradigm to **text tokens**: represent each text token as $K_t$ codes from $K_t$ text codebooks (with $K_t \leq K$ for text). This is motivated by the observation that word meanings are compositional—a text codebook could separately encode morphology, syntax, and semantics, analogous to how visual codebooks separately encode color, texture, and structure. The $K_t$ text codes would be produced by a learned text tokenizer that decomposes each subword into $K_t$ factored representations, and the same autoregressive codebook head would predict them. This creates true structural symmetry between text and visual modalities: both are sequences of multi-codebook tokens decoded by the same depth-autoregressive mechanism. The alignment benefit is that the backbone no longer needs to maintain separate representational strategies for text (single code $\to$ logits) vs. vision (multi-code $\to$ depth-AR logits). This connects to the "text as a diffusion target" perspective from DiffusionGemma (Review #107) and AURORA-LM (Review #108), which used continuous latent spaces for text diffusion; multi-codebook text provides a discrete alternative that preserves the unified masked-diffusion framework. The factored text representation could also benefit continual learning (T3): new knowledge could be added to a dedicated "semantic" codebook while "syntactic" codebooks remain frozen, echoing Cortex's dual plasticity principle.

**Feasibility:** Medium. Factoring text tokens into multiple codebooks requires designing a text tokenizer with meaningful decomposition axes, which is non-trivial. A simpler starting point: apply random linear projections to split each text embedding into $K_t$ subspaces that are then discretized and treated as codebook entries. The main risk is that text tokens already carry rich semantic information in single codes; the multi-codebook decomposition may not provide the same benefit as in the visual domain where spatial information is inherently multi-faceted.

## 3 Conclusion

Intern Lumina U2 demonstrates that multi-codebook discrete tokenization unlocks dramatically higher representational capacity for visual tokens within diffusion LLMs, achieving $4096^8$ possible states per spatial position without increasing sequence length. The key architectural innovations—FSQ-based 4D tokenization, depth-autoregressive prediction head, and task-aware attention masking—enable a single masked-diffusion backbone to handle text, images, video,

and 3D under one objective. The model improves understanding on 8 of 10 benchmarks over the same-backbone LLaDA2.0-Uni while maintaining generation quality, and the PHV sampling technique provides a practical 4.45× inference speedup. For diffusion LLM research, the paper establishes the multi-codebook paradigm as a viable path to closing the quality gap between discrete diffusion and continuous generation approaches. The depth-autoregressive factorization introduces a principled way to model within-position dependencies, creating a new architectural dimension (codebook depth) that is orthogonal to the sequence (spatial) dimension. Future work connecting adaptive codebook allocation to the asymmetric compute principles from looped architectures, and extending multi-codebook representations to text, could further unify and streamline multimodal diffusion language models.

## References


[1] Q. Yu et al. Intern Lumina U2: A Multi-Codebook Diffusion Large Language Model for Omni-Visual Understanding and Image Generation. *arXiv preprint arXiv:2610.20818*, 2026.

[2] S. Nie et al. LLaDA2.0-Uni: Unifying Multimodal Understanding and Generation with Diffusion Large Language Model. *arXiv preprint arXiv:2604.20796*, 2026.

[3] Z. Author et al. SpecFold: Folding Multi-Branch Redundancy for Faster Speculative Decoding in Diffusion Language Models. *arXiv preprint arXiv:2610.04875*, 2026.

[4] Z. Author et al. SoloQ: Calibration-Free Quantization for Diffusion Language Models. *arXiv preprint arXiv:2610.07121*, 2026.

[5] J. Zhang et al. Cortex: Consolidating Context into Parametric Memory. *arXiv preprint arXiv:2610.20399*, 2026.